

Create a clean v1.5.48 production release file.

1.2

Recommendation: in v1.5.48, Section F should validate number/date format only when the dashboard column definition has `dateColumn === true`. Non-date text columns should not trigger default number-format failures.

Recommendation: change the governed default behavior for non-date template data cells to text ("@") or, better, do not validate non-date number format in Section F unless a column explicitly declares a special number format.

1.4

Recommendation: for v1.5.48, Raw Data needs a governed template-application function that applies the Template - Raw Data formatting to the existing active Raw Data output sheet without leaving source/duplicate Raw Data sheets behind.

1.5

It is dashboard-driven, but it is still multi-step and range-operation-heavy. The spec's One Pass Processing concept is more naturally aligned to data/business processing, but the template/formatting layer should still be optimized toward a "single resolved formatting plan" and minimal batched writes.

2. Performance Bottlenecks

2.1

Recommendation: in v1.5.48, skip row-resize logic entirely when `rowCount <= 1000`. Google Sheets already creates new sheets with 1000 rows, so those templates should not pay any row-resize cost.

2.2

For Monthly Change, verify subsection title row, spacer row, subsection header row, and formatting block.

Validate date format only for dashboard columns where `dateColumn === true`.

If no issues exist, write one PASS summary row.

2.3 - the formatting is linked to the column headers and should be the same across all templates.

Recommendation: for v1.5.48, build a resolved template-formatting plan in memory first, then apply it in larger, predictable blocks. Avoid repeated column-by-column validation and formatting where possible.

3. Architecture Issues

3.1

The current checked-in script is still a previous release artifact and should not be patched again.

3.2 create a separate governed formatting stage:--- What would this do? With the Raw Data why can't it's Formatting use the same path as the other sheets. Create copy of template, copy and paste data to sheet.

Recommendation: create a separate governed formatting stage:

3.3

Recommendation: split validation responsibilities:

Section F: template existence, headers, required rows, Monthly Change subsections, date-format columns only.

Formatting diagnostics / optional QA: widths, hidden columns, default text format, row heights, filters.

Performance report: separate timing by first build, full rebuild, metadata-only refresh.

3.4 Monthly Change subheader structure still needs final output validation

The script has a Monthly Change subheader guide function and a spacer formatting function.

However, Section F currently does not show a complete dedicated validation sequence for:

subsection title row;

spacer/dark row;

subsection header row;

copied headers in 8pt font;

formatting block.

The user's visual requirement is specific: after the dark/spacer row, the next row should copy the headers in 8pt font. That needs to be validated by Section F and verified in final output.

4. Prioritized Remediation Plan

Priority 1 — Create v1.5.48 as a full production release, not a patch

Do this first.

Create Current_Production/v1.5.48_Production_Script.txt.

Set MASTER_LIST_MERGE_ML_VERSION = "1.5.48".

Do not keep modifying v1.5.47.1.

Do not deliver patch snippets as the production artifact.

This directly resolves the versioning and governance issue.

Priority 2 — Fix Section F validation scope

Section F should be rewritten or heavily simplified to match the stated requirements:

template exists;

title rows exist where required;

header row exists;

expected headers match dashboard definitions;

missing headers identified;

unexpected headers identified;

required subsections validated;

Monthly Change subsection title/spacer/header/format block validated;

date format validation only when dashboard dateColumn === true;

one PASS row if clean.

Remove default "General" validation from Section F. The current code is still validating all expected headers against number formats, causing false failures.

Priority 3 — Make text the governed non-date default

The script already has defaultTextFormat: "@", but it is not the active fallback for non-date format validation.

Recommended behavior:

date columns: validate/apply mm/dd/yyyy or dashboard date format;

explicitly numeric columns: validate/apply explicit dashboard number format;

ordinary non-date text columns: apply text format where needed but do not fail Section F because Google Sheets reports a non-identical default display format.

Priority 4 — Build a real Raw Data template-application path

Raw Data currently calls a function that intentionally does not apply template formatting.

For v1.5.48, add a governed function such as:

`applyTemplateFormattingToExistingOutputSheet_()`

It should:

use Template - Raw Data;

apply template formatting to the existing final Raw Data sheet;

keep only one final formatted sheet;

not leave a source Raw Data sheet and separate formatted copy;

apply widths, header/title/data styles, hidden columns, date/text formats, freeze/filter where governed.

Priority 5 — Remove row-resize work for templates with 1000 rows or less

The script still calls row resizing during every full template build.

The helper avoids deleting extra rows but still does work.

For v1.5.48:

if `rowCount <= 1000`, do not call row resize at all;

do not record a row-resize timing step for those templates;

only add rows when dashboard requires more than 1000 rows.

This should directly reduce the Create Templates timing bottleneck.

Priority 6 — Rebuild templates/formatting architecture before improving processing

Yes — for templates and formatting, I recommend rebuilding the Create Templates path cleanly, but not rebuilding business processing yet.

The rebuild should be limited to:

Dashboard config loading.

Resolved global/default formatting.

Template creation/refresh.

Template-to-output formatting.

Dashboard Quality Section F.

Timing/report write behavior.

Do not rebuild Demo P, Master List, Care Plan, Banners, Monthly Change business logic yet. Stabilize QA/Templates/Formatting first, then modularize.

Direct Answer: Is templates/formatting using One Pass Processing?

No.

The current template builder is dashboard-driven but still performs multiple formatting passes and multiple Apps Script range operations: title rows, header row, data rows, widths, number formats, hidden columns, filters, subheaders, and row heights are all applied separately.

For v1.5.48, the better architecture is:

resolve all dashboard/default formatting into one in-memory plan;

write template values and formats in the fewest practical range operations;

validate only governed structural/date requirements in Section F;

apply template formatting to output sheets through one governed template-application function.

```
git fetch origin --prune
```

```
git checkout -B work origin/main
```

```
git status -sb
```

```
git log -1 --oneline
```

```
find . -maxdepth 2 -type f | sort | head -300
```

Copy

Startup / Sync Verification Complete

I ran the requested commands and did not modify files.

Command Results

✓ git fetch origin --prune

Fetches latest origin/main.

origin/main advanced from 67ce8fd to 6a96240.

✓ git checkout -B work origin/main

Reset local work branch to origin/main.

Branch work is tracking origin/main.

✓ git status -sb

Current status: ## work...origin/main

No working-tree changes shown.

✓ git log -1 --oneline

Current commit: 6a96240 Commit

✓ find . -maxdepth 2 -type f | sort | head -300

Confirmed current key files include:

AGENT.md

README.md

Current_Production/v1.5.47.1_Production_Script.txt

Current_Production/v1.5.47_Production_Script.txt

Report/v1.5.47.1 - Dashboard Quality Report (1).pdf

Report/v1.5.47.1 - Framework Timing Report (1).pdf

Report/v1.5.47.1 - Google Apps Scripts Executions (1).pdf

spec/v1.9_Master_List_Framework_Specifications_Final_Governing_Edition.pdf

spec/v1.4.26_Production_Script_Archive.txt

spec/v1.4.26.1_Production_Script_Archive.txt

src/Code.gs

Repository State

Branch: work

Tracking: origin/main

Current commit: 6a96240

Remote: <https://github.com/tabs912/ML.git>

Sync status: up to date with origin/main

Working tree: clean

Governance Notes

AGENT.md confirms the framework must maintain Single File Architecture, Dashboard Driven Framework, Dashboard Controlled Templates, Template First Formatting, Primary PMR Architecture, One Pass Processing, Dashboard Quality Architecture, Timing Framework, and Validation Framework.

README.md confirms the framework remains in Production Stabilization and should not introduce modular architecture without governance approval.

README.md also confirms the expected One Pass Processing model is Read Once → Process In Memory → Write Once.

Missing Artifact Check

No required top-level startup artifacts are missing:

AGENT.md present.

README.md present.

Current_Production/ present.

spec/ present.

Requested (1) report files are now present under Report/.

Readiness status: ready for next instruction.